



LedgerProof Protocol v1.1 — Security Audit Engagement Briefing

Document: LPR-AUDIT-SOW-REQ-2026-06 **Prepared by:** LedgerProof Foundation (in formation) / LedgerProof Inc. **Recipient:** NCC Group Cryptography Services (primary) / Trail of Bits (alternate) / Cure53 (verifier-scope alternate) **Date:** 2026-06-01 **Target kickoff:** 2026-06-08 **Target memo publication:** 2026-08-31 **Confidentiality:** Pre-engagement; share under standard mutual NDA

1. ENGAGEMENT SCOPE STATEMENT

LedgerProof is an open, Bitcoin-anchored cryptographic protocol that produces tamper-evident receipts of AI-deployment-governance events. The receipts are designed to discharge EU AI Act Article 50 (transparency obligations on deployers of generative-AI systems) evidentiary burden by giving a CRO/CCO/regulator a third-party-anchored, independently verifiable proof that a governance event (deployment, review, mitigation, incident, retirement) occurred at a point in time with a specific reviewer assertion attached.

We are commissioning a **fixed-fee, fixed-scope security audit of protocol v1.1**, the production release that has been live since 2026-05-18 and currently anchors approximately 50,000 receipts to the Bitcoin mainnet via OP_RETURN.

1.1 What you are being asked to evaluate

1. The cryptographic core: canonicalization (Rust + TypeScript), Ed25519 signing path, SHA-256 entry hashing, Merkle tree construction, Bitcoin OP_RETURN payload format.
2. The chain integrity model: append-only entry chain, `prev_hash` linkage, anchor-to-block linkage, witness-path verification.
3. The verifier (Vite/TypeScript SPA at `verify.ledgerproofhq.io`): correctness, strict-mode signature verification, supply-chain posture of `@noble/ed25519` and `@noble/hashes`.
4. The slug-router authorization model (`aliases.json` , `pre-v1-entries.json`) — the mutable indirection layer that a Foundation insider could in theory abuse.
5. LPR-ERRATA-001 root-cause analysis: independent confirmation that the Entry #0 `content_hash` mismatch is a pre-v1.0 publisher draft artifact, that verifier code is correct, and that the defect is contained to Entry #0.

1.2 What you are NOT being asked to evaluate

- The Bitcoin network itself, the SCITT IETF draft semantics, or the EU AI Act legal interpretation.
- Operational security of LedgerProof Inc. corporate IT (separate engagement).
- Foundation board governance (counsel scope).
- Anything in the legacy `iad` deployment (3 anchors, frozen, untouchable — declared out of scope by design; receipts remain verifiable forever and are not modified).

1.3 What constitutes a finding

Any deviation from the documented protocol specification (/04-lpr-spec), any cryptographic primitive misuse, any divergence between Rust and TypeScript canonicalization on a valid input, any path by which a non-key-holder can produce a receipt that the production verifier accepts, any path by which a key-holder can equivocate without detection, any supply-chain vector that would allow signature-verification bypass in the verifier, and any GDPR-relevant data leak through the schema.

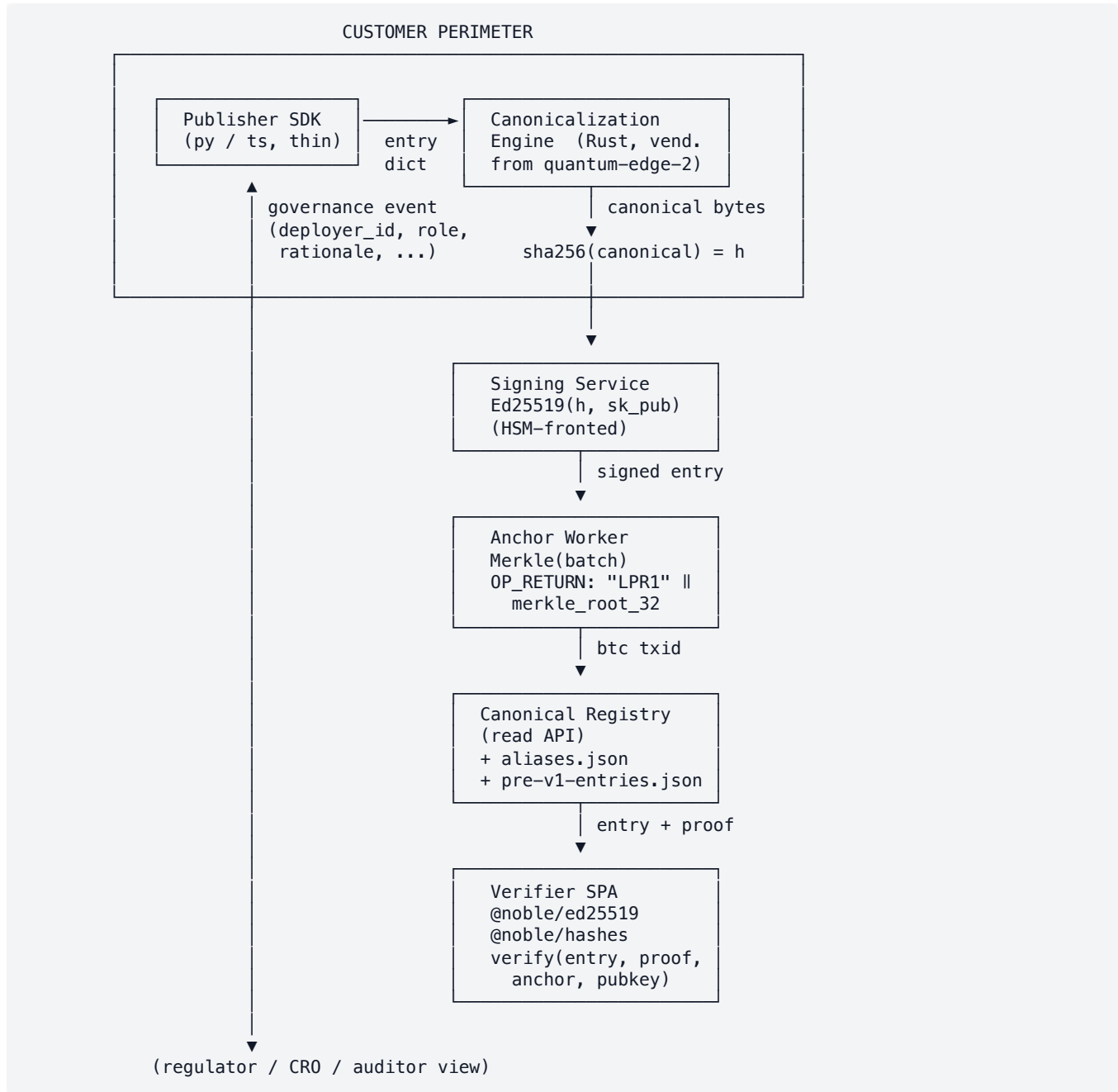
1.4 Deliverable

A signed 5–10 page memo, published verbatim at `security.ledgerproofhq.io/<firm>/2026-08-<scope>` , severity-classified per § 7, with reproduction steps and a sign-off on the LPR-ERRATA-001 RCA.

1.5 Success

We can hand the memo to a Tier-1 EU CRO/CCO buyer, to BaFin, to AFM, and to the EU AI Office, and have it withstand technical scrutiny.

2. SYSTEM ARCHITECTURE OVERVIEW



2.1 Components, trust boundaries, key material

#	Component	Language	Trust boundary	Inputs	Outputs	Key material	Persisted state
1	Publisher SDK	Python 3.11+, TypeScript (ESM+CJS)	Customer trust domain	Governance event dict	Canonical entry dict	None	None (stateless)
2	Canonicalization Engine	Rust (vendored from quantum-edge-2)	LedgerProof Inc. trust domain	Entry dict	Canonical byte sequence + SHA-256 digest	None	None
3	Signing Service	Rust	LedgerProof Inc. trust domain (HSM-fronted)	SHA-256 digest	Ed25519 signature	Publisher signing keys (HSM-resident)	Signing audit log
4	Anchor Worker	Rust	LedgerProof Inc. trust domain	Batch of signed entries	Bitcoin txid + Merkle proofs	BTC anchor key (HSM-resident, segregated)	Merkle tree, anchor ledger
5	Canonical Registry	Rust (axum), Postgres	Foundation read trust	Entry id / slug	Entry + Merkle proof + anchor metadata	None (read path)	Entry store, <code>aliases.json</code> , <code>pre-v1-entries.json</code>
6	Verifier SPA	TypeScript (Vite)	Public trust domain (runs in browser)	Entry + proof + anchor txid + pubkey	Verification result	Public keys (pinned + fetched)	None (stateless)

The **Foundation root key** is a separate trust artifact, ceremony-generated Day 90 (~2026-08-15), stored under an m-of-n threshold; it signs the publisher key set and `aliases.json` revisions.

3. ASSETS AT RISK

Asset	Class	Where it lives	Why it matters
Publisher signing keys (Ed25519 sk)	Confidentiality + Integrity	HSM (Inc.)	Compromise = arbitrary receipt forgery
Foundation root key	Confidentiality + Integrity	m-of-n threshold ceremony, offline	Compromise = full PKI rotation
Merkle roots	Integrity	In-memory at anchor time; on-chain forever	Anchor binds receipts to time
OP_RETURN payloads	Integrity	Bitcoin mainnet	The notarization anchor
Canonical JSON byte sequences	Integrity	Transient (recomputed on verify)	Hash agreement is the protocol's keystone
Customer governance content	Confidentiality (GDPR)	Schema-rejected at SDK boundary	Schema must reject emails in <code>deployer_id</code> , <code>reviewer_role</code> , <code>review_rationale</code>
<code>aliases.json</code>	Integrity + Availability	Registry repo; Foundation-controlled	Mutable indirection layer; insider risk surface
<code>pre-v1-entries.json</code>	Integrity	Registry repo; Foundation-controlled, append-only	Pre-v1.0 exception register; LPR-ERRATA-001 lives here
Signing audit log	Integrity (non-repudiation)	Inc.-controlled, WORM-tier	Forensic basis for incident response

4. THREAT MODEL — STRIDE per Component

Severity scale used below: **L** = Low, **M** = Medium, **H** = High, **C** = Critical. Likelihood: **L/M/H** independently. "Residual" = our assessment of residual risk after current mitigation; the auditor is asked to dispute these.

4.1 Publisher SDK (Component 1)

STRIDE	Threat	Actor	Like.	Impact	Current mitigation	Residual	Audit verification
S	Spoofed publisher submits forged event upstream of signing	External	M	H	Mutual TLS + API key + per-deployer rate limit	M	Review API auth; attempt token replay
T	SDK mutates the event after schema validation but before canonicalization	Supply chain	L	C	Canonicalization is bitwise reproducible; verifier recomputes	L	Differential test: SDK output vs. independent canonicalizer
R	Customer denies submitting an event	Customer	M	M	Signed API receipt + WORM audit log	L	Inspect API receipt format
I	PII leakage via reviewer_role / deployer_id / review_rationale	Customer (accidental)	H	H	Schema regex rejects RFC 5322 email-like patterns	M	Fuzz schema with international email forms, unicode homoglyphs, base64-wrapped PII
D	Schema-validation DoS via pathological input (deeply nested JSON)	External	M	M	Depth + size caps in SDK	L	Submit nested-object bombs
E	SDK-to-signing service privilege confusion	External	L	H	SDK has no signing key	L	Confirm capability boundary

4.2 Canonicalization Engine (Component 2) — CRITICAL FOCUS

This is the keystone. **Bitwise hash agreement across implementations is the single most load-bearing property of the protocol.**

STRIDE	Threat	Actor	Like.	Impact	Current mitigation	Residual	Audit verification
T	Rust and TypeScript produce different canonical bytes for the same input	Implementation bug	M	C	lpr-test-vectors v0.2.0 corpus (100 entries) cross-validated	M	Extend corpus with adversarial vectors (see § 6); attempt to find divergent hash
T	Unicode normalization mismatch (NFC vs NFD vs NFKC) on review_rationale	Implementation bug	M	C	Spec mandates NFC; both impls normalize before hashing	M	Submit pre-NFD, pre-NFKC, mixed-form strings; verify byte equality
T	Number representation drift (1 vs 1.0 , 1e2 vs 100 , -0 vs 0)	Implementation bug	M	C	Spec mandates integer-or-decimal canonical form; NaN/Inf rejected at schema	M	Number-edge-case fuzz; confirm rejection of NaN/Inf/-0
T	Key ordering divergence in nested objects	Implementation bug	L	C	Lexicographic byte-order, recursive	L	Nested-object permutation suite
T	Trailing whitespace / BOM / control character handling	Implementation bug	M	H	Schema rejects control chars; canonicalization strips none (rejects instead)	L	Inject U+FEFF, U+200B, U+2028
I	Canonicalization side-channel reveals customer content	Internal	L	L	Engine is offline-pure; no logging of plaintext	L	Code review for log statements

4.3 Signing Service (Component 3)

STRIDE	Threat	Actor	Like.	Impact	Current mitigation	Residual	Audit verification
S	Unauthorized signing request	Insider	L	C	HSM-gated; per-request authorization token; mTLS	L	Attempt unauthorized sign; review HSM config
T	Signature over wrong digest (digest substitution)	Insider	L	C	Signing service recomputes digest from canonical bytes it received	M	Confirm digest-recomputation; attempt substitution
R	Insider denies signing an entry	Insider	L	H	WORM audit log; HSM signing counter	L	Reconcile HSM counter to log
I	Private key extraction	Insider / supply chain	L	C	HSM-resident, non-exportable; FIPS 140-3 Level 3	L	Confirm HSM attestation
D	Sign queue overload	External	M	M	Rate-limit per-tenant; backpressure	L	Load test
E	Service-account escalation to root-key signing	Insider	L	C	Root key segregated to Day 90 ceremony m-of-n	L	Review IAM segregation

4.4 Anchor Worker (Component 4)

STRIDE	Threat	Actor	Like.	Impact	Current mitigation	Residual	Audit verification
T	Merkle tree construction off-by-one (duplicate-last-leaf attack à la CVE-2012-2459)	Implementation bug	M	C	RFC 6962-style domain separation (<code>0x00</code> leaf, <code>0x01</code> internal); odd-leaf handling spec'd as "promote, do not duplicate"	M	Construct adversarial tree; attempt second-preimage
T	OP_RETURN payload is not exactly 36 bytes (<code>"LPR1"</code> + 32)	Operator error / impl bug	L	H	Anchor worker asserts <code>len == 36</code> before broadcast; node would reject non-standard size in many cases	L	Force 37/35-byte payloads in test harness
T	Two batches share a Merkle root by collision/coincidence	External	negligible	C	SHA-256 preimage resistance	L	Cryptographic argument, not test
R	Anchor worker equivocates: same entry in two Merkle trees with different surrounding context	Insider	L	H	<code>prev_hash</code> chain linkage; entry index monotonic; anchor ledger	M	Attempt fork: produce two valid receipts for same entry index
I	Anchor metadata reveals customer batch composition	External	M	L	Only Merkle root is published; entry order randomized within batch	L	Review batch policy
D	Bitcoin mempool fee spikes block anchoring	External	H	M	Pre-funded fee buffer; CPFP fallback	M	Review fee policy
E	Anchor key compromise → arbitrary OP_RETURN broadcasts	Insider	L	H	HSM-gated; segregated from signing	L	Confirm key segregation

Reorg-specific case: the protocol requires **6-confirmation finality** before an anchor is reported as `confirmed`. If a block is orphaned past 6 confirmations (statistically negligible on mainnet but not zero), the receipt re-enters `pending`; the entry is re-anchored in the next batch. The auditor should validate the re-

anchoring code path and confirm it does not produce a second valid receipt with a different anchor while the first is still publicly cached.

4.5 Canonical Registry (Component 5) — SECOND CRITICAL FOCUS

STRIDE	Threat	Actor	Like.	Impact	Current mitigation	Residual	Audit verification
T	<code>aliases.json</code> re-points <code>/r/founding-declaration</code> to a different entry	Foundation insider	M	C	<code>aliases.json</code> revisions signed by Foundation root key (m-of-n); revision history append-only; verifier displays the underlying entry id, not just the slug	M	Drill: simulate insider with one root-key share; confirm m-of-n threshold prevents unilateral re-pointing; confirm verifier UI exposes underlying entry id
T	<code>pre-v1-entries.json</code> allowlist abused to whitelist a forged "legacy" entry	Foundation insider	L	H	File is append-only in git; root-key signed; verifier checks ed25519 sig over file before honoring exception	M	Drill: attempt to add a <code>pre_v1</code> entry post-hoc; confirm verifier rejects without valid root signature
R	Registry serves different content to different verifiers (split-view)	Insider	L	H	Verifier should re-derive integrity from on-chain anchor, not trust registry	M	Run two verifier instances against tampered registry; confirm both detect tampering via anchor
I	Registry leaks customer content via API enumeration	External	M	M	Schema already filters PII; entry ids are random	L	Enumerate API
D	Registry DoS denies verifier resolution	External	M	M	CDN-fronted; static cacheable	L	Load test
E	Read-API actor escalates to write	Insider	L	C	Strict IAM split	L	IAM review

The auditor MUST drill on the `aliases.json` and `pre-v1-entries.json` insider-risk path. The mitigating principle is: *the slug-router is a UX convenience; cryptographic integrity flows from entry id → canonical bytes → SHA-256 → Ed25519 sig → Merkle root → OP_RETURN, and never through a slug.* The auditor's job is to confirm that the verifier honors this principle and that no code path elevates the slug above the entry id in trust.

4.6 Verifier SPA (Component 6) — THIRD CRITICAL FOCUS

STRIDE	Threat	Actor	Like.	Impact	Current mitigation	Residual	Audit verification
S	Phishing site mimics verifier and reports false "verified"	External	M	H	CSP, COOP/COEP, SRI; reproducible build; published SHA-256 of bundle	M	Review CSP; confirm SRI on @noble/*
T	Malleable Ed25519 signature accepted	Implementation bug	M	C	@noble/ed25519 strict mode (RFC 8032 §5.1.7 small-order/cofactor checks)	M	Submit malleable / non-canonical S; confirm rejection
T	Witness path tampering on Merkle inclusion proof	External	M	C	Verifier walks proof bottom-up; rejects on length mismatch / sibling-order violation	M	Submit malformed witness: wrong length, swapped sibling order, duplicate-last-leaf, intermediate-node-as-leaf
T	Anchor txid points to OP_RETURN with wrong magic bytes	External	M	H	Verifier checks <code>payload[0:4] == "LPR1" AND len == 36</code>	L	Submit txids with wrong magic
T	Pubkey substitution via tampered key bundle	External / supply chain	M	C	Pubkeys signed by Foundation root key; verifier pins root key fingerprint in source	M	Attempt key substitution; confirm fingerprint pin holds
I	Verifier leaks query targets via 3rd-party fetch	External	M	L	All fetches first-party; no analytics; no fonts	L	Review network panel
E	XSS in verifier promotes attacker pubkey	External	L	C	No HTML rendering of entry fields; everything text-escaped	L	Standard XSS suite

Supply chain — @noble/ed25519 and @noble/hashes : - Pin exact versions (no ^, no ~); commit `package-lock.json` . - Publish SHA-256 of the npm tarball in the audit memo and at `security.ledgerproofhq.io/integrity` . - Build the verifier in a reproducible CI pipeline; publish the bundle hash; SRI-pin from the HTML. - Auditor is asked to confirm the published hashes match what is actually served.

GDPR Article 17 (right-to-erasure) semantics: The chain is append-only. "Erasure" is implemented as a soft-delete tombstone: the entry's content is redacted in the registry response, but the entry's hash, signature, anchor, and `prev_hash` linkage remain. The verifier displays "REDACTED PER GDPR ART. 17" and still verifies the chain. The auditor should confirm that erasure does **not** break chain verification for any entry, and that the tombstone itself cannot be forged.

5. PRIOR KNOWN ISSUES — LPR-ERRATA REGISTER

LPR-ERRATA-001 — Entry #0 stored `content_hash` mismatch.

- **Root cause:** Pre-v1.0 publisher draft serialized one field with a trailing newline that v1.0 canonicalization correctly strips. Entry #0 was anchored under the pre-v1.0 form; its stored `content_hash` reflects the pre-v1.0 bytes; the v1.0 verifier, recomputing from canonical bytes, would diverge.
- **Containment:** Bug is in the *publisher* draft, not the *verifier*. Confined to Entry #0. Entries 1+ verify correctly under v1.0.
- **Remediation:** The chain is append-only; Entry #0 is enshrined. `pre-v1-entries.json` records Entry #0 with the pre-v1.0-byte form as the basis of comparison, signed by the Foundation root key.
- **What we ask the auditor to validate:** 1. RCA is correct (the pre-v1.0 byte form deterministically reproduces the stored hash). 2. No other entry is affected. 3. The `pre-v1-entries.json` allowlist mechanism cannot be exploited to whitelist a *forged* legacy entry (see § 4.5). 4. The verifier's branch that dispatches to pre-v1.0 comparison for entries listed in the allowlist is correct and cannot be triggered for entries 1+.

6. TEST VECTORS THE AUDITOR INHERITS

`lpr-test-vectors v0.2.0` — 100 entries, Rust + TypeScript cross-validated, MIT-licensed, available at github.com/ledgerproof/lpr-test-vectors. Each vector is `{ input: <event dict>, canonical_bytes: <hex>, sha256: <hex>, signature: <hex>, pubkey: <hex> }`.

The auditor is expected to:

1. **Reproduce.** Run both reference implementations against the corpus and confirm bitwise hash agreement on all 100 vectors.
2. **Extend adversarially.** Add at minimum: - Unicode normalization edge cases (NFC/NFD/NFKC/NFKD pairs that map to the same NFC). - Number representations (`-0`, `0.0`, `1e0`, `1.0e2`, `100`, `1.0`, very large integers near 2^{53} , negative-zero exponents). - NaN / Inf / subnormal floats (schema must reject). - Nested-object key permutations (3+ levels deep). - Empty arrays, empty objects, null in optional fields. - U+FEFF (BOM), U+200B (ZWSP), U+2028/U+2029 (line/paragraph separators). - Strings exactly at the schema length limit, one over, one under. - Surrogate-pair edge cases (lone surrogates must reject).
3. **Attempt divergence.** Try to find a single input that produces different SHA-256 digests across Rust and TypeScript. *This is the highest-value finding possible in this engagement.*
4. **Optional but encouraged:** Write a third reference implementation in Go and re-run. We will accept it into the test-vector repo under the auditor's copyright + MIT license.

7. EXPECTED DELIVERABLES

1. **5–10 page signed memo**, published at `security.ledgerproofhq.io/<firm>/2026-08-<scope>`.
2. **Severity-classified finding list** using this taxonomy: - **Critical** — receipt forgery, signature bypass, canonicalization divergence, key extraction - **High** — chain-integrity break for non-erased entries, slug-router insider abuse path - **Medium** — supply-chain risk, schema gap, operational hardening - **Low** — defense-in-depth - **Informational** — observations, suggestions
3. **Reproduction steps** for every finding (PoC code where applicable).
4. **Coordinated disclosure** via `security@ledgerproofhq.io`; 90-day default, accelerated if Critical.
5. **Explicit sign-off** on LPR-ERRATA-001 RCA or a counter-RCA.

6. **Test-vector contribution** (adversarial vectors added to the corpus).

8. SOW BUDGET FRAMEWORK

Fixed-fee, milestone-billed. Indicative ranges:

Scope	Effort	Range (USD)
Canonicalization-only audit	3 days	\$15K – \$25K
Full v1.1 protocol audit + supply chain	3 weeks	\$80K – \$120K
Verifier-specific (Cure53 typical)	2 weeks	\$60K – \$90K
Combined: full v1.1 + verifier (our preferred scope)	5–6 weeks	\$150K – \$200K

Budgeted ceiling: **\$200K** including PoC development and one round of remediation re-test.

9. ENGAGEMENT TIMELINE

Date	Milestone
2026-06-08	Kickoff; NDA + SoW executed; repository access granted
2026-06-15	Architecture walkthrough; threat-model alignment session
2026-06-22	Test-vector reproduction complete; first findings if any
2026-07-13	Mid-engagement readout; preliminary findings shared
2026-08-10	Draft memo delivered; remediation begins
2026-08-24	Remediation re-test
2026-08-31	Final memo signed and published

The publication date is hard. The first EU AI Act Article 50 enforcement examination cycle opens in September 2026; we need this memo in regulator hands before then.

10. POST-AUDIT REMEDIATION PROCESS

1. **Triage (within 48h of finding):** LedgerProof CTO (Veronica S. Dawkins) + auditor lead jointly assign severity. Critical findings pause the next release.
2. **Ownership:** Each finding is assigned a single owner from the protocol-engineering team. Tracked in `lpr-spec` repo as a GitHub issue with the `audit-2026-08` label.
3. **SLA:** Critical = 7 days to patch; High = 14 days; Medium = 30 days; Low = next release; Informational = backlog.
4. **Re-test:** Auditor verifies remediation; sign-off recorded in the published memo as an addendum.
5. **Public disclosure:** Coordinated; CVE issued where applicable.
6. **Foundation reporting:** Quarterly summary to the Foundation board (Veronica + 2 independent directors); annual public transparency report.

Primary contact (engineering): Veronica S. Dawkins, CTO — `vsd@ledgerproofhq.io` / PGP fingerprint published at `/.well-known/security.txt` **Primary contact (security disclosure):** `security@ledgerproofhq.io` / PGP same source **Repository access:** granted on SoW execution; read on `lpr-spec`, `lpr-canonicalization`, `lpr-anchor`, `lpr-registry`, `lpr-verifier`, `lpr-test-vectors` **Preferred bid format:** Reply to this document section-by-section; deviations from scope flagged explicitly; fixed-fee with milestone billing.

— end of briefing —

Document boundary: This document is internal strategic planning material for LedgerProof Foundation (501(c)(3) in formation, Delaware) and LedgerProof Inc. (Delaware C-corp). It contains assumptions, forecasts, and decisions that may change as conditions evolve. Numbers labeled BASE / STRETCH / DOWNSIDE are scenarios, not forecasts. Distribution is limited to Foundation board, Inc. officers, and named diligence counterparties under NDA or equivalent.

Confidentiality: Do not redistribute without consent of the founder. Bitcoin-anchored governance receipts apply only to ratified Foundation actions; this document is a planning artifact and is NOT receipt-anchored.